

Kernel Assessment for High Dimensional Bayesian Optimization

Abhigya Koirala¹, Andrew Yuan², and Jimmy Risk³

¹Kenyon College

²Cooper Union

³Cal Poly Pomona

August 9, 2025

Abstract

Bayesian Optimization (BO) is a popular method for optimizing black-box functions, but there has been a long-standing belief in the literature that it struggles in high-dimensional settings. In this work, we assess the performance of various covariance kernels in a variety of high dimensional synthetic and real world optimization tasks, assisting with recent work that challenges the common notion that standard Gaussian Processes (GPs) fail high-dimensional settings. As opposed to previous works that solve this problem by simply adding priors or appropriately initializing hyperparameters, we study the impact of kernel choice and compare it to the existing methods. We study 12 kernels across 6 synthetic and real world benchmark problems. Our results show that the Generalized Cauchy (GCauchy) kernel performs well and consistently without relying on Automatic Relevance Determination (ARD), robust initialization, or priors on high dimensional BO tasks. This suggest kernel choice alone is sufficient, and sometimes even better for competitive performance in high dimensions.

1 Introduction

Bayesian Optimization (BO) is a strategy to find global optimum of black-box functions over some input space that are typically noisy and expensive to evaluate. As these functions are almost always unknown, the aim of BO is instead to build a probabilistic model of the objective function under a limited sampling budget. BO is comprised of two main components: a probabilistic surrogate (commonly a Gaussian Process (GP)) and an acquisition function. Rather than trying to model the objective function as a whole, the acquisition function decides the most informative points for the BO algorithm to find global optimum as efficiently as possible.

Despite its uses, BO has typically been considered to be ineffective in high dimensions (> 20 dimensions). This has prompted a lot of research on specialized high-dimensional BO techniques (see Table 1) which simplify the problem by imposing strong structural assumptions. However, recent work has shown that standard GP based BO can perform well in high dimensions, especially with proper lengthscale initialization or use of priors. These results imply that the limitation of ordinary BO may be due to poor implementation rather than an inherent property of high-dimensional GPs.

While recent work to address the failure modes for SBO in HDBO focus on initialization strategies and applying priors, we focus a more fundamental aspect of the GP: kernel choice. We speculate that certain kernels beyond the traditional Matérn class can combat some effects like gradient

vanishing in high dimensions due to e.g. heavy tailed decay characteristics or flexibility in other areas. This work investigates the impact of different standard GP kernels on high dimensional BO. We evaluate the kernels’ performance and behavior on a variety of synthetic functions and real world optimization tasks.

Our major contributions are summarized as follows:

- **Highlight the role of kernel choice to avoid gradient vanishing** We provide empirical results of parsimonious and expressive kernels that drastically improve BO efficiency, showing how the Generalized Cauchy kernel maintains gradients during training, improving optimization stability.
- **Visualize kernel effect on BO in low dimensions** We provide visualizations on how different kernels provide different GP surrogates and influence the acquisition function over time in easily visualizable 1D and 2D problems.
- **Demonstrate usefulness in high dimensional settings** We assess 12 kernel methods across six real world and synthetic problems with up to 300 dimensions, showing which kernels scale best.
- **Analyze hyperparameter convergence over optimization steps** We visualize the trend of the GP hyperparameters (particularly lengthscale, and α, β for Generalized Cauchy) across BO iterations to show patterns in stability and convergence behavior for each kernel.

Paper Outline

Section 2 provides some background on GPs, kernels and an overview of BO. Section 3 provides a literature study on recent work for SBO in HDBO. Section 4 presents experimental results, and Section 5 discusses implications and future directions.

2 Background and Problem Setup

2.1 Gaussian Process for Surrogate Modeling

A Gaussian process (GP) f over a set $\mathcal{X}$ is a stochastic process such that for any n and $\{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subseteq \mathcal{X}$ we have the vector $[f(\mathbf{x}_1), \dots, f(\mathbf{x}_n)]^\top$ is multivariate normal. They are characterized through their first two moments, offering a finite-dimensional structure for the entire infinite-dimensional distribution of f . The *mean function* and the *covariance kernel* of a GP are defined as

$$\mu(\mathbf{x}) \triangleq \mathbb{E}[f(\mathbf{x})], \quad (1)$$

$$k(\mathbf{x}, \mathbf{x}') \triangleq \text{cov}(f(\mathbf{x}), f(\mathbf{x}')), \quad (2)$$

where $k(\mathbf{x}, \mathbf{x}')$ is a symmetric and positive definite function. We use the shorthand $f \sim \mathcal{GP}(\mu, k)$ to denote that f is a Gaussian process with mean function μ and covariance kernel k . The mean function determines the overall trend of the process, while its covariance captures the dependencies between different inputs and how these dependencies affect the outputs. Different choices of mean and covariance functions lead to different properties of the corresponding GP. The Gaussian process regression model uses f to learn an input-output relationship between $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^d$ and the latent

output $\mathbf{x} \mapsto f(\mathbf{x}) \in \mathbb{R}$. The response mapping f is not known, and is accessed through some *samples* y 's. Given a *training dataset*

$$\mathcal{D} \triangleq \{(\mathbf{x}_i, y_i) : i = 1, \dots, N\}, \quad (3)$$

the idea is to decompose y_i into an underlying signal $f(\mathbf{x}_i)$ and noise $\epsilon(\mathbf{x}_i)$, whereby the observed $\mathbf{y} \triangleq [y_1, \dots, y_N]^\top$ provide data-driven inference for $f(\mathbf{x}_*)$ for any $\mathbf{x}_* \in \mathcal{X}$ by the conditioning equations. This is formalized as a noisy data generating process $\mathbf{x} \mapsto y(\mathbf{x})$

$$y(\mathbf{x}) = f(\mathbf{x}) + \epsilon(\mathbf{x}). \quad (4)$$

We assume $(\epsilon(\mathbf{x}))_{\mathbf{x} \in \mathcal{X}}$ is a collection of independent mean-zero random variables with variance $\sigma^2(\mathbf{x}) \geq 0$. The GP regression assumption is that the underlying “signal” f follows a Gaussian process, that is, $f \sim \mathcal{GP}(\mu, k)$. We think of this GP as the *prior* (with *prior mean* μ and *prior covariance kernel* k), i.e. the default assumption imposed by the modeler before seeing data. The goal of the regression is then to obtain the posterior $f(\mathbf{x}_*)|\mathcal{D}$ for a test point $\mathbf{x}_* \in \mathcal{X}$, or more broadly, $f|\mathcal{D}$ (the posterior process). This corresponds to conditioning f on the observed data. Importantly, the probability space that describes sample outcomes of f is *independent* of the observation noise ϵ . In the literature, one distinguishes the former *extrinsic* or aleatoric uncertainty about f from the latter *intrinsic* uncertainty due to ϵ . A conditioning argument provides that the posterior (predictive) $f(\mathbf{x}_*)|\mathcal{D}$ is also a Gaussian process, where for a single test point $\mathbf{x}_*$ the predictive mean and variance are

$$m(\mathbf{x}_*) = K(\mathbf{x}_*, \mathbf{X}) [\mathbf{K}_f + \Sigma_\epsilon]^{-1} \mathbf{y}, \quad (5)$$

$$s^2(\mathbf{x}_*) = k(\mathbf{x}_*, \mathbf{x}_*) - K(\mathbf{x}_*, \mathbf{X}) [\mathbf{K}_f + \Sigma_\epsilon]^{-1} K(\mathbf{X}, \mathbf{x}_*). \quad (6)$$

Similarly, we obtain that $y(\mathbf{x}_*)|\mathcal{D} \sim \mathcal{N}(m(\mathbf{x}_*), s^2(\mathbf{x}_*) + \sigma^2(\mathbf{x}_*))$, so the variance of a new observation $y(\mathbf{x}_*)$ is decomposed into the intrinsic randomness $\sigma^2(\mathbf{x}_*)$ and the extrinsic uncertainty $s^2(\mathbf{x}_*)$.

2.2 Positive Definite Kernels

A kernel is a covariance function that defines the similarity measure between two inputs and determines certain properties in the prior such as smoothness and periodicity. Since a constant mean is often assumed in Gaussian process models, the choice of kernel becomes the primary factor that characterizes the GP and governs its dynamics. In Automatic Relevance Determination (ARD), each input dimension is assigned a lengthscale hyperparameter which enables the model to identify the relevance of each dimension.

We list the kernels used in our experiment down below:

- Squared Exponential (SE):

$$k_{SE}(x, x'; \ell_{\text{len}}) = \exp\left(-\frac{\|x - x'\|^2}{2\ell_{\text{len}}^2}\right)$$

- Matérn Class:

$$k_{\text{Matérn}}(x, x'; \nu, \ell_{\text{len}}) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(r\sqrt{\frac{2\nu}{\ell_{\text{len}}}}\right)^\nu K_\nu\left(r\sqrt{\frac{2\nu}{\ell_{\text{len}}}}\right)$$

In practice ν is fixed where typically $\nu = 1/2 + p$ for $p = 0, 1, 2, \dots$ such that the kernel function has a closed form.

- Rational Quadratic Kernel:

$$k_{\text{RQ}}(x, x') = \left(1 + \frac{\|x - x'\|^2}{2\alpha\ell^2}\right)^{-\alpha} = \left(1 + \frac{\rho^2}{2\alpha}\right)^{-\alpha}, \quad \rho = \frac{\|x - x'\|}{\ell}$$

- Generalized Cauchy Kernel:

$$k_{\text{GCauchy}}(x, x') = \left(1 + \left(\frac{\|x - x'\|}{\ell}\right)^\alpha\right)^{-\beta/\alpha} = (1 + \rho^\alpha)^{-\beta/\alpha}, \quad \rho = \frac{\|x - x'\|}{\ell}, \quad \alpha \in (0, 2], \beta > 0$$

If $\beta \in (0, 1]$ it has long memory with Hurst coefficient $H = \beta/2$. If $\alpha = 2$, then General Cauchy reduces to the Rational Quadratic with some reparameterization.

The kernel hyperparameters are typically learned by maximizing the marginal log likelihood of the training data. This process adjusts the covariance structure so the model best fits the data.

2.3 Bayesian Optimization Overview

The following is a brief overview on Bayesian Optimization. Specific details may be omitted; refer to Frazier et al. [1] for further clarification.

As mentioned previously, BO is a method used to find global optimum of expensive and noisy black-box functions with a limited sampling budget. Starting with an initial space of points, the acquisition function selects new points to be iteratively evaluated and refit the GP prior with the new data until the sampling budget is exhausted as shown in Algorithm 1.

A GP prior is placed onto the unknown objective function f :

$$f \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$$

We use the GP regression model as a cheap surrogate. At iteration n of the algorithm, we have a probabilistic model providing the mean $m(\mathbf{x})$ and variance $s^2(\mathbf{x})$ at any candidate input $\mathbf{x}$. An acquisition function then leverages these $(m(\mathbf{x}), s^2(\mathbf{x}))$ pairs to determine the utility of a point in the input parameter space. By maximizing the acquisition function, balancing between exploration and exploitation, BO determines the next best point to query. Common choices for acquisition functions include Expected Improvement and Upper Confidence Bound (UCB) due to their easy-to-implement heuristic and stable performance. To keep consistent with Xu et al. [2], we implement UCB as the acquisition function in our experiment. The UCB acquisition function features explicit exploitation vs exploration terms, defined as :

$$\alpha_{\text{UCB}}(x; \lambda) = m(x) + \lambda s(x)$$

where λ is a weight parameter representing exploration level.

Algorithm 1 Outline of Standard Bayesian Optimization Algorithm

Place a Gaussian process prior on f by choosing $\mu(\mathbf{x})$ and $k(\mathbf{x}, \mathbf{x}')$
 Evaluate f at n_0 points on some parameter space of potential points H .
 Initialize training set $\mathcal{D}$ with these n_0 points and set $n = n_0$
while $n \leq N$ **do**
 Update the GP to be $f(\cdot)|\mathcal{D}$, the posterior conditioned on $\mathcal{D}$
 Let $x_n \in H$ be a maximizer of the acquisition function based on $f(\cdot)|\mathcal{D}$
 Evaluate $y_n = f(x_n)$
 Append (y_n, x_n) to $\mathcal{D}$
 Increment n
end while
return the best point according to the optimization objective

2.4 High-Dimensional Bayesian Optimization

There is a widely held belief that BO becomes ineffective in high dimension, the threshold being around 15 to 20 dimensions. This belief is rooted in the *curse of dimensionality*— as dimension d increases, the input domain vastly increases, rendering data points to be more sparse. In particular, it becomes increasingly more difficult to model covariance structures properly as distances grow, where the average distance between two randomly selected points in a d -dimensional hypercube is proportional to $\sqrt{d}$ [3]. To achieve an accurate model for f in these contexts, more sampling points are required which increases model complexity and complicates acquisition function optimization more challenging due to the expanded search space and flatter acquisition landscape [4].

3 Related Work

Recent work has showed that standard GP-based BO can still excel in high dimensions once its failure modes are properly addressed.

3.0.1 Gradient Vanishing from Improper Initialization

In particular, Xu et al. (2025) [2] focus on *gradient vanishing*, explaining how poorly initialized kernel lengthscales cause gradients of the marginal likelihood maximization to tend to 0, stalling model learning (see Appendix B). This issue especially arises when using the widely adopted Squared Exponential (SE) kernel. To address this, they propose two alternative solutions: (1) a robust initialization (RI) of the lengthscale prior ($\ell_0 = c\sqrt{d}$), or (2) replacing the SE kernel with a Matérn kernel.

3.0.2 Dimension-Dependent Lengthscale Priors

Similarly, Hvarfner et al. (2024) points out that standard lengthscale priors place substantial density on low lengthscale values to consider all dimensions equally. This traditional assumption leads to practical difficulties in high dimensional settings, where data points are more sparsely distributed, making global modeling infeasible. To address this, Hvarfner et al. (2024) propose a dimension-scaled prior, *DSP*, which the kernel lengthscales are drawn from a log-normal prior whose mean and mode explicitly scale with input dimension. This adjustment decreases model complexity while preserving the model’s ability to maintain meaningful correlations between distant observations. Their method is evaluated against other specialized HDBO baselines across three categories of

benchmark functions and optimization tasks: Sparse Synthetic, Mid-dimension (10-30D), and High Dimension (>120D). In all cases, SBO with DSP achieves competitive or superior performance with reduced runtime.

3.1 Benchmarking and Evaluation

To compare the performance of standard BO (SBO), both Xu et al. (2025) [2] and Hvarfner et al. (2024) [5] benchmark their proposed “fixes” against other specialized HDBO methods listed in Table 1 across several synthetic functions and real world optimization tasks (see Appendix A).

Table 1: Classification of BO Algorithms

Category	Benchmark Methods	Source/Citation
Classical Bayesian Optimization	SBO-Matérn, SBO-Matérn-RI, SBO-SE, SBO-SE-RI	[2]
	Vanilla-BO	[5]
Specialized High Dimensional Algorithms	TURBO-1, TURBO-5	[6]
	HESBO-10, HESBO-20	[7]
	ALEBO-10, ALEBO-20	[8]
	SAASBO-NUTS, SAASBO-MAP	[9]
	RDUCB	[10]
	TREE UCB	[11]
	Bounce	[12]
	MCTS-VS	[13]
	MPD	[14]
	CMA-ES	[15]

Xu et al. (2025) [2] reports that with RI, SBO often outperforms specialized HDBO methods such as *Tree UCB* and *RDUCB*. Without proper initialization, however, the SE kernel performs poorly, particularly when the dimension exceeds 150, as opposed to the Matérn kernel which achieved top-tier performance across all benchmark methods, except where the dimension exceeded 1,000, verifying its robustness. Likewise, in Hvarfner et al. (2024), SBO with DSP achieves competitive or superior performance with reduced runtime against specialized HDBO baselines across three categories of benchmark functions and optimization tasks: Sparse Synthetic, Mid-dimension (10-30D), and High Dimension (>120D).

The results of both studies show that, with a few targeted adjustments, standard BO remains robust across a diverse function structures. They offer computational efficiency and are easy to implement by relying on maximum likelihood estimation instead of expensive MCMC sampling, and do not rely on complex or unintuitive structural assumptions. However, Xu et al. (2025) finds a substantial performance difference between the SE and Matérn kernel in high dimensional settings. The Matérn kernel performs superior and is more robust to the issue of gradient vanishing, questioning the significance of kernel choice as well. These findings motivate our exploration of different kernel choices and their performances in HDBO.

4 Experiments

In this section, we aim to investigate whether kernel choice can alone solve known difficulties in HDBO. We evaluate different GP kernels in BO across diverse high dimensional synthetic and real world benchmark tasks. We proceed in two parts. The first is to give a visual aid for how kernel choice can impact the BO algorithm, by visualizing the surrogate and the BO process for different GP kernels in 1D and 2D settings. The second, which is the main goal of the paper, assesses kernel performance in high dimensional functions.

4.1 Low-Dimensional Visualization Experiment

For the sake of intuition on how GP kernels work on the surrogate and the BO process, we visualize the evolution of the GP posterior and the acquisition function’s decisions on low dimensional problems using four characteristic test functions: the non-smooth 1D and 2D Ackley function (non-differentiable at its minimum), the smooth 1D Griewank function, and the smooth 2D Booth function. The 1D Griewank function is defined as:

$$f(\mathbf{x}) = \sum_{i=1}^d \frac{x_i^2}{4000} - \prod_{i=1}^d \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$$

with global minimum at $\mathbf{x} = \mathbf{0}$ and domain $\mathcal{X} = [-600, 600]$.

The 2D Booth function is defined as:

$$f(\mathbf{x}) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$$

with global minimum at $\mathbf{x} = \mathbf{0}$ and domain $\mathcal{X} = [-10, 10]^2$. We compare four kernels: Matérn-1/2, Matérn-5/2, Rational Quadratic, and Generalized Cauchy.

4.1.1 Implementation Details

For each function and dimension, we start with 5 initial points, sampled using LHS, and perform 15 BO iterations. The surrogate is trained for 1000 epochs with a 0.1 learning rate. For the BO iterations, we warm restart for 50 epochs with a 0.1 learning rate. For each case, we show the results at different steps in the BO process: after initialization, 1/3 through, 2/3 through, and at the final step. To control for randomness, we utilize one random seed.

4.1.2 Results and Visualizations

We visualize the GP posterior mean and confidence intervals for the 1D functions over time and present the contour plot of the true function surface alongside overlaid chosen points at different steps of BO for the 2D case.

Non-Smooth Function (Ackley) Figure 1 demonstrates how kernel choice affects the GP surrogate model throughout BO for 1D Ackley. Across iterations, Matérn-5/2 oversmooths the function and fails to model the non differentiable structure near the minimum, while Matérn-1/2 being less smooth adapts better. The Rational Quadratic kernel performs moderately, providing a good balance between smoothness and local variation. The Generalized Cauchy kernel struggles in the beginning but does well eventually in modeling the global structure, causing more accurate posterior estimates and sampling near the optimum.

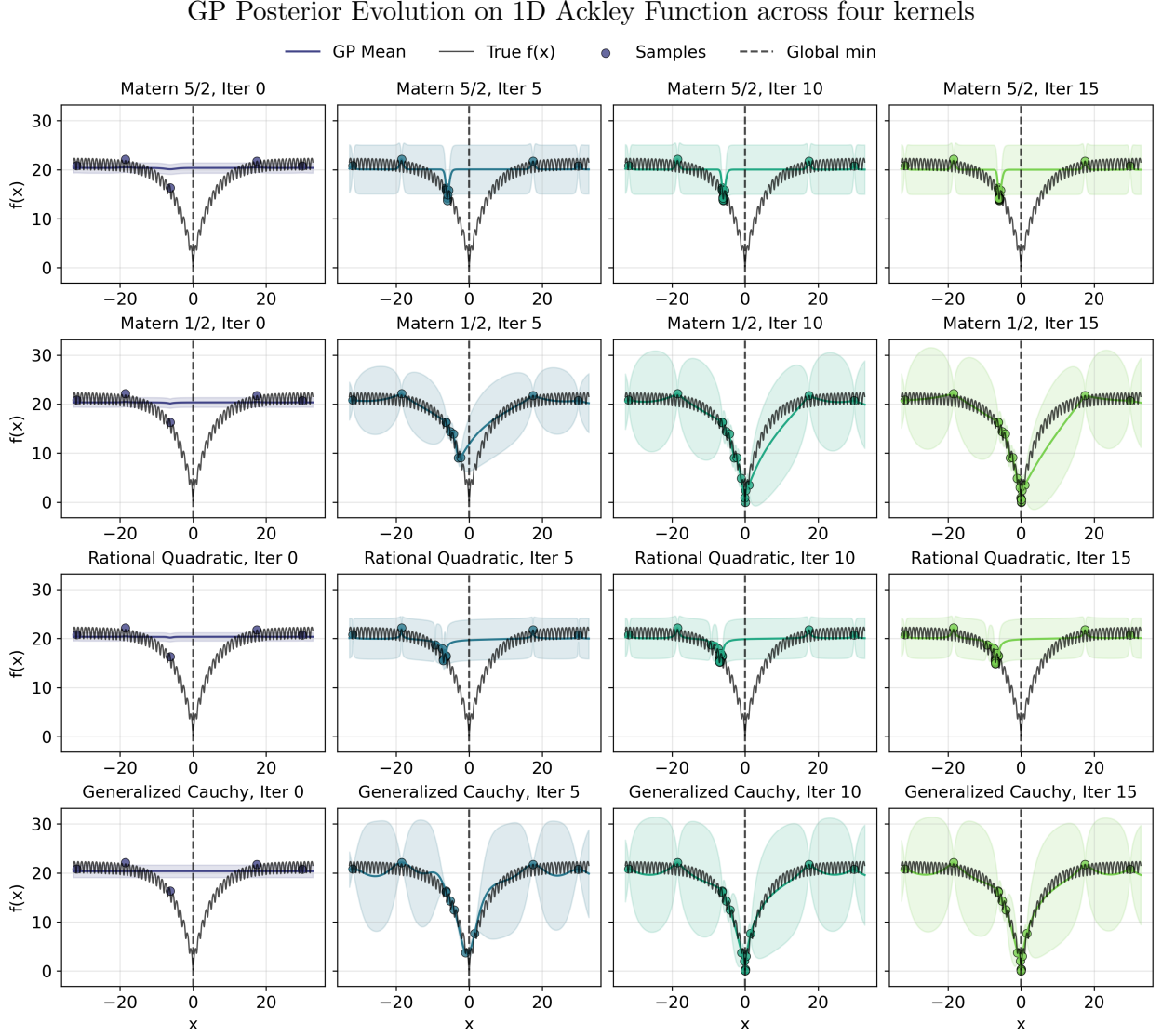


Figure 1: Evolution of GP surrogate models during Bayesian Optimization on the 1D Ackley function using four different kernels: Matérn-5/2 (top row), Matérn-1/2 (second row), Rational Quadratic (third row), and Generalized Cauchy (bottom row). Each row shows posterior updates at iterations 0, 5, 10, and 15. The true objective is shown in black, the GP mean in blue, confidence intervals as shaded regions, and sampled points as dots. The known global minimum at $x = 0$ is marked with a vertical dashed line. Smoother kernels like Matérn-5/2 struggle to adapt to the non smooth minimum, while less smooth kernels like Matérn-1/2 and Generalized Cauchy better approximate the true function and guide sampling effectively.

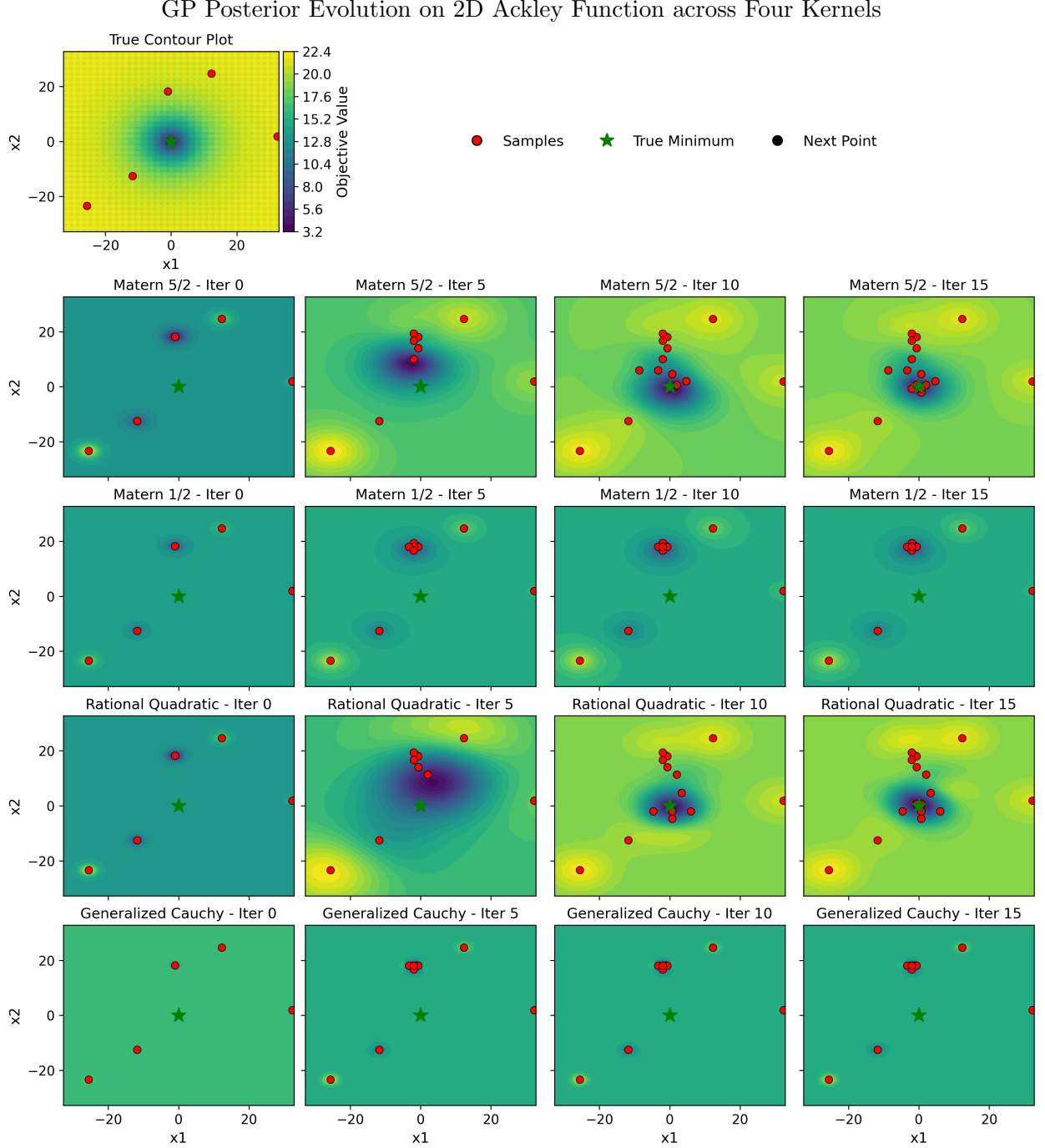


Figure 2: Bayesian Optimization on the 2D Ackley function using Gaussian Process surrogates with four different kernels: Matérn-5/2 (second row), Matérn-1/2 (third row), Rational Quadratic (fourth row), and Generalized Cauchy (fifth row). Each kernel’s performance is visualized at iterations 0, 5, 10, and 15. The top-left plot shows the true objective contour with sampled points (red), the known global minimum (green star), and axis labels x_1 and x_2 . All other plots show the GP posterior mean contours, updated at each iteration, along with sampled points and current acquisition suggestions (black cross). Matérn-5/2 adapts quickly getting closer to the minimum but on the other hand, Matérn-1/2 and Generalized Cauchy kernels are still exploring and perform worse. We attribute this to the number of BO iterations being on the lower end and expect GCauchy (and Matérn-1/2) to perform well as the iterations go on, as seen in Ackley 1D.

Smooth Functions (1D Griewank and 2D Booth) We repeat the experiments done on the 1D and 2D Ackley functions for the 1D Griewank and 2D Booth functions. Figure 3 presents the evolution of the GP posterior in 1D over BO iterations. Matérn-5/2 performs better than Matérn-1/2 due to its smoother prior assumptions aligning better with the nature of the function. Rational Quadratic performs the best in learning the structure and exploring effectively, showing least variability across all four kernels. Generalized Cauchy, despite being more flexible, converges slower early on but catches up by the final iterations.

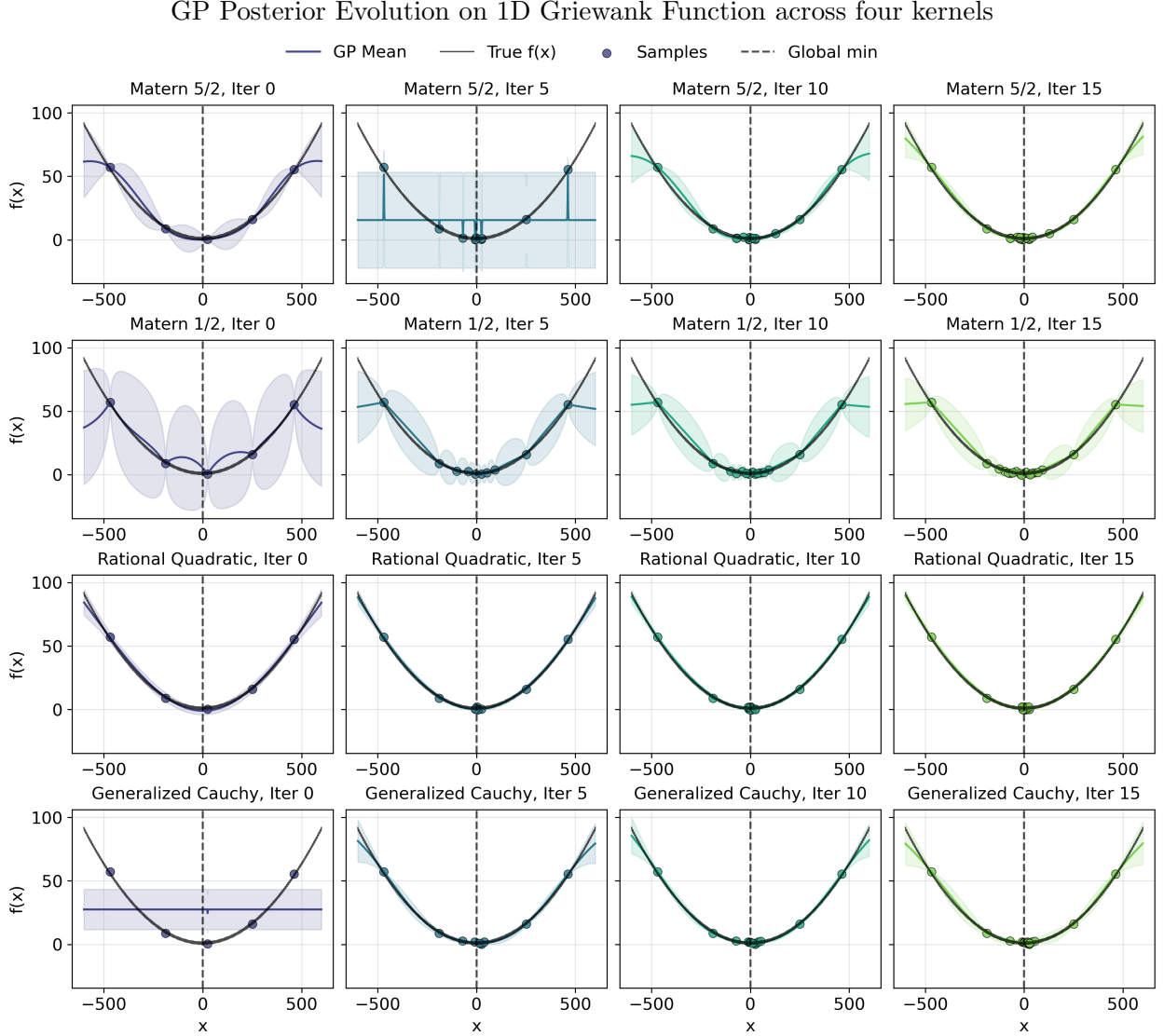


Figure 3: GP surrogate evolution over Bayesian Optimization iterations on the 1D Griewank function. Shown are the posterior means and uncertainties for Matérn-5/2 (top row), Matérn-1/2 (second row), Rational Quadratic (third row), and Generalized Cauchy (bottom row) at iterations 0, 5, 10, and 15. The true function is shown in black. All kernels do an excellent job on getting to the minimum and predicting the underlying structure.

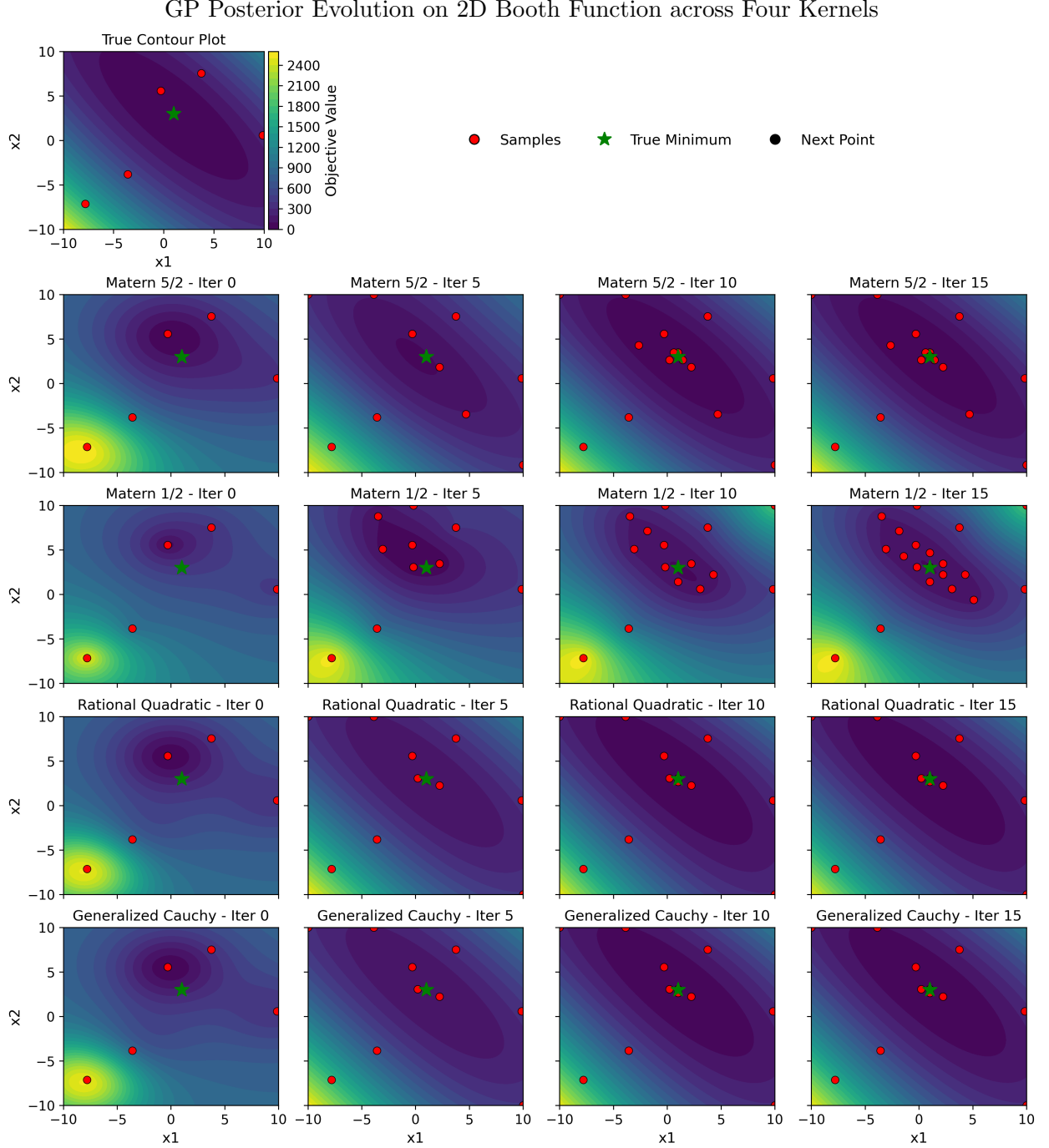


Figure 4: Bayesian Optimization using GP surrogates on the 2D Booth function. Each row (Matérn-5/2, Matérn-1/2, Rational Quadratic, and Generalized Cauchy) shows GP posterior mean contours at iterations 0, 5, 10, and 15. The true function contour is shown in the top-left for comparison. Red dots indicate sampled points, green stars mark the true minimum, and black points show the next acquisition suggestion. The kernels don't model the surrogate very effectively but all kernels except Matérn-1/2 fix this and get to the minimum effectively towards the end of the BO process.

Figure 4 shows the results for the 2D Booth function. Here, all kernels other than Matérn-1/2 do a good job in getting close to the minimum. Matérn-1/2 suffers from the non differentiable condition imposed and fails to effectively model the underlying function and get as close as the other kernels to the optimum.

4.2 High-Dimensional Benchmark Functions Experiment

We evaluate the performance of different GP kernels in BO across six diverse benchmark functions present in Xu et al. (2025) [2], closely following their experiments for consistency and a fair comparison. We benchmark on their methods on both synthetic test functions and real world black box problems. All tasks are maximization problems by default.

We include the "Standard BO" method from Xu et al. (2025)'s [2] as a comparison to our regular BO across different kernel choices. For succinctness, we omit a direct comparison with specialized HDBO methods and instead benchmark our approach against their results, indirectly comparing to the broader class of specialized HDBO algorithms. For reference, specific performance metrics of the specialized HDBO methods are available in Xu et al. (2025) [2].

Table 2 categorizes the benchmark problems used in our experiments. More details about them can be found in the appendix A.

Table 2: Classification of Benchmark Test Functions	
Category	Benchmark Functions (d, d')
Synthetic Test Functions	Rosenbrock (100,100)
	Rosenbrock (300,100)
	StybTang (200,200)
Real-world with constraints	Mopta08 (124)
	Rover (60)
	DNA (180)

4.2.1 Implementation Details

When utilizing GPs as surrogate models in SBO, we consider the effects of the outputscale σ_f^2 and additive noise (or nugget) σ^2 hyperparameters on model flexibility and stability.

Xu et al. [2] adopt the default method in BoTorch to handle hyperparameters, which involves setting a prior. Utilizing L-BFGS, Adam, and RMSProp optimizers, they maximize the log marginal likelihood of the GP regression model, ensuring length-scale and noise variance parameters stay positive via the SoftPlus transform. To achieve numerical stability with the L-BFGS optimizer, Xu et al. imposes a prior Uniform(0.001, 30) over each length-scale, a diffused Gamma prior over the noise variance Gamma(1.1, 0.05), and the prior Gamma(2.0, 0.15) over the signal amplitude, terminating optimization after 15,000 iterations or when objective-change falls below 2E-9.

In our experiments, we set the outputscale σ_f^2 to be a fully learnable hyperparameter in addition to fixing the nugget to 0 since our motive is to evaluate each kernel's capacity to capture the true signal. In addition, our method differs from previous work in three key ways:

Table 3: Experiment Implementation Settings

	Our Implementation	Xu et al. (2025) Implementation
Lengthscale Prior	None (use MLE)	Uniform(0.001, 30)
Outputscale Prior	None (use MLE)	Gamma(2.0, 0.15)
Nugget (Noise Var.)	$\sigma^2 \in [10^{-16}, 10^{-2}]$ (use MLE)*	Gamma(1.1, 0.05)
Optimizer	Adam	L-BFGS

- **No Priors:** Our approach relies purely on maximum likelihood estimation to fit the hyperparameters, as opposed to the Maximum A Posteriori (MAP) approach used in Xu et al. (2025) or the Log Normal ($\mathcal{LN}$) prior placed on each lengthscale ℓ_i in Hvarfner et al. (2024).
- **Constrained Noise:** The observation noise variance is optimized via maximum likelihood, but is constrained to a small interval, $\sigma^2 \in [10^{-16}, 10^{-2}]$, acting as a numerical regularizer rather than a real observation noise. This matches the idea that the sampled values are not noisy.
- **Optimizer:** We use Adam ($\eta = 0.1$) instead of L-BFGS to handle the highly non-convex likelihood surface.

In keeping consistent with Xu et al. [2], we use the Upper Confidence Bound acquisition function with the exploration λ fixed at 1.5 to balance exploration and exploitation. The GP surrogate is modeled using GPyTorch, while the BO loop and acquisition function optimization are done using BoTorch.

Kernels and Variants

In total, we ran twelve kernels across the benchmark tasks listed in Table 2 in our experiment.

Among the four kernels listed below, the two RI variants are from Xu et al. (2025) and represent their proposed robust initialization.

- **SBO-Matérn:** Matérn-5/2-ARD kernels with the length scale initialization $\ell_0 = \text{SoftPlus}(0) = 0.693$ by default GPyTorch and BOTorch practices.
- **SBO-Matérn (RI):** Matérn-5/2-ARD kernels with robust initialization $\ell_0 = c\sqrt{d}$ where d is the input dimension and $c = 1$ as proposed in Xu et al. [2].
- **SBO-SE:** SE-ARD with the length scale initialization $\ell_0 = \text{SoftPlus}(0) = 0.693$ by default GPyTorch and BOTorch practices.
- **SBO-SE (RI):** SE-ARD kernels with robust initialization $\ell_0 = c\sqrt{d}$ where d is the input dimension and $c = 1$ as proposed in Xu et al. [2].

We ran a selection of common and more exotic kernel choices against the RI-based variants proposed by Xu et al. (2025) to see if potential kernels could outperform Xu et al.’s proposed robust initialization.

- **SBO-Matérn:** Matérn-1/2 and Matérn-5/2 with the length scale initialization $\ell_0 = \text{SoftPlus}(0) = 0.693$ by default GPyTorch and BOTorch practices.

- **SBO-Matérn (RI)**: Matérn-1/2 and Matérn-5/2 kernels with robust initialization $\ell_0 = c\sqrt{d}$ where d is the input dimension and $c = 1$ as proposed in Xu et al. [2].
- **SBO-GCauchy**: Generalized Cauchy kernel
- **SBO-GCauchy (RI)**: Generalized Cauchy kernel with robust initialization where $c = 1$.
- **SBO-RQ**: Rational Quadratic kernel
- **SBO-RQ (RI)**: Rational Quadratic kernel with robust initialization where $c = 1$.

Bayesian Optimization Procedure

To run BO, we randomly select 20 initial input points from the target function using Latin Hypercube Sampling (LHS). The surrogate is first trained for 2000 epochs using the Adam optimizer with a learning rate of 0.1. In subsequent BO iterations, we use a *warm restart* training strategy where the GP model and optimizer states are initialized from the previous state and the model is retrained for 50 epochs per iteration, with the same learning rate. Furthermore, for robustness, each experiment is repeated 10 times with different random seeds (0 through 9), drawing a new set of 20 initial points in each trial.

4.2.2 Results and Discussion

Kernel Performance Across Dimensions Quantitative results on optimization performance by kernel and benchmark function are presented below. We compare several kernels, including Matérn, RQ, SE, and GCauchy with and without RI. Each violin plot summarizes the distribution of the best objective found across repeated BO runs.

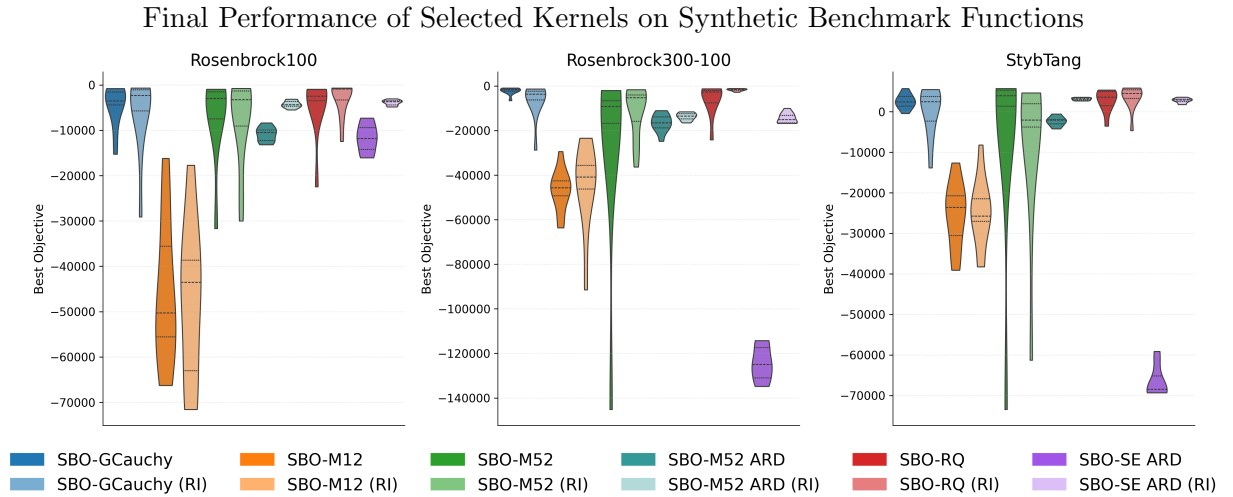


Figure 5: Comparison of kernel performance on synthetic benchmark functions as mentioned in Table 2. Each violin plot displays the distribution of the best objective values achieved across repeated BO runs for different kernels.

We examined the optimization performance for each kernel across the synthetic and real-world benchmark problems using violin plots that illustrate the average of the obtained maximum function value over the 10 seeds (the middle bar), the best (the top bar), the worst (the bottom bar), and the distribution of the obtained maximum (the shaded region).

We first discuss kernel performance across the synthetic benchmark tasks: the GCauchy and RQ kernels consistently obtains the highest objective value with both RI and non-RI kernel variant across all input dimensions. Compared Matérn-1/2 and Matérn-5/2, kernels, the GCauchy and RQ kernels not only matches but also surpasses these widely used kernels. In particular, the GCauchy exhibits low variability across an array off seeds, highlighting its reliability in common BO scenarios where repeated simulations are impractical. This behavior is most prominent in the Rosenbrock300-100 scenario where the generalized Cauchy kernel well outperforms the SBO-M52-ARD, SBO-M52 (RI), SBO-SE-ARD, and SBO-SE-ARD (RI) kernels, including the methods that Xu et al. [2] propose. Even without implementing *robust initialization*, the success of the RQ and more impressively, the GCuachy kernel, shines a light on the impact of kernel choice’s in model performance.

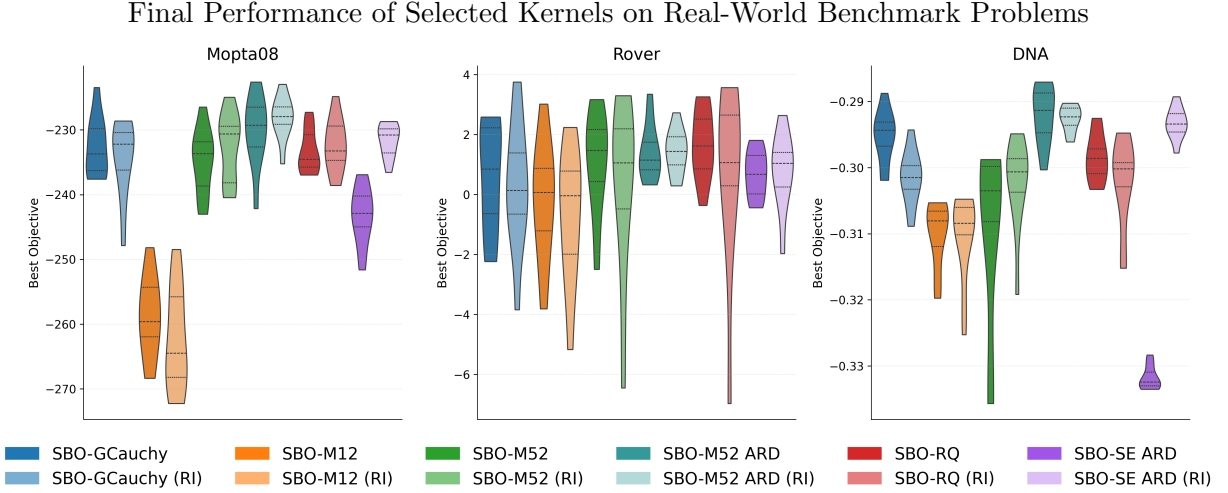


Figure 6: Comparison of best kernel performance on real world benchmark functions as mentioned in Table 2. Each violin plot displays the distribution of the best objective values achieved across repeated BO runs using different kernels. Differences between kernels are less noticeable in these noisy and lower dimensional (in comparison to the synthetic benchmark problems) settings. However, kernels like Matérn-5/2 with ARD and General Cauchy consistently perform well. The SE ARD (RI) kernel’s underwhelming performance on DNA highlights its sensitivity to the problem structure.

Across real-world benchmark functions, the GCauchy and RQ kernels are not as advantageous, but still maintains competitive performance with the best-performing kernels. The GCauchy kernel again outperforms Matérn 1/2 and Matérn 5/2 with RI especially for Mopta08 and DNA by a wide margin. Even in Rover where GCauchy’s median performance is slightly lower it exhibits significantly reduced spread compared to standard Matérn-1/2 and Matérn-5/2 kernels, including the RI variants. Notably, GCauchy kernel also consistently outperforms the SE kernel, even with RI proposed by Xu et al. Similarly, the RQ kernel surpasses the SE-ARD (RI) kernel except in the DNA optimization task. These results further verify the limitations of the SE kernel in high-dimensional settings, as its performance remains inferior to the GCauchy kernel even without RI. The shortcomings of the SE kernel to match the RQ and GCauchy kernel, despite implementing fixes from Xu et al. suggests an unsuitability for high-dimensional problems.

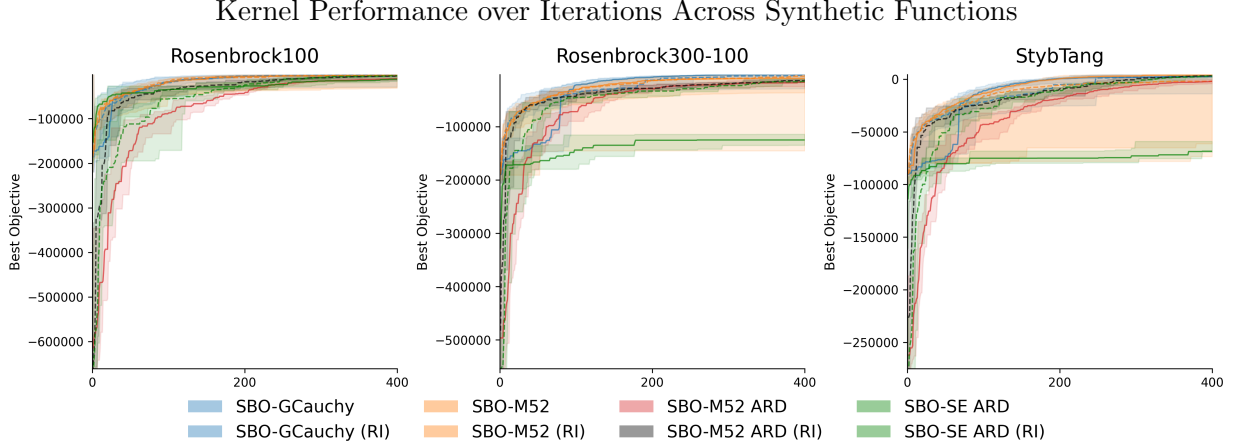


Figure 7: Comparison of the optimum value obtained over iterations for each kernel on synthetic benchmark problems as mentioned in Table 2. We plot the median, best, and worst performance over iterations of the BO algorithm for a subset of kernel configurations. For a cleaner view, we omit standard baselines and non-ARD variants select kernels, focusing on the comparison between GCauchy and Matérn-5/2 and SE kernels with ARD and RI as proposed in Xu et al.

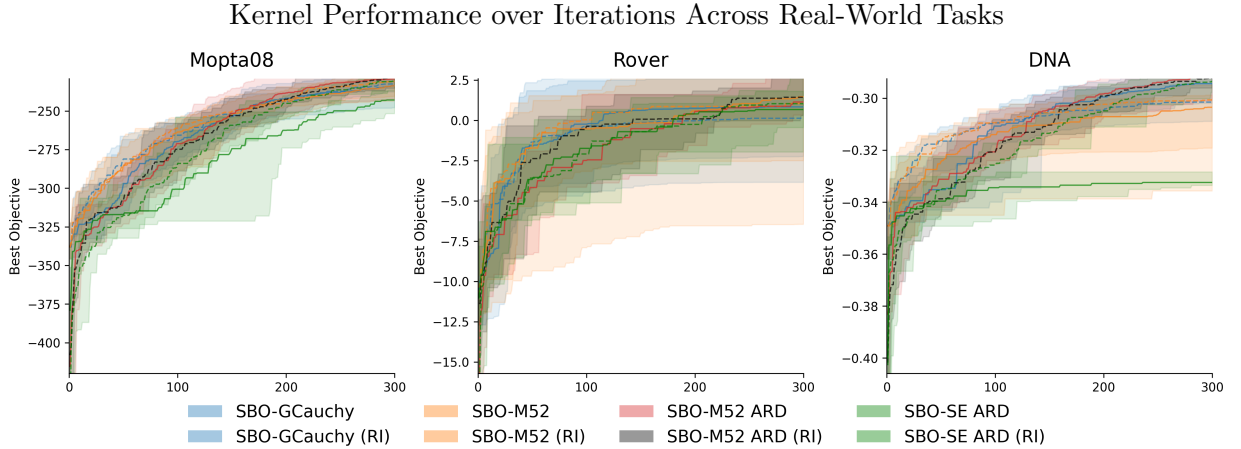


Figure 8: Comparison of the optimum value obtained over iterations for each kernel on real-world benchmark functions as mentioned in Table 2. We plot the median, best, and worst performance over iterations of the BO algorithm for a subset of kernel configurations. For a cleaner view, we omit standard baselines and non-ARD variants select kernels, focusing on the comparison between GCauchy and Matérn-5/2 and SE kernels with ARD and RI as proposed in Xu et al.

4.2.3 Hyperparameter Analysis

We analyze key GP hyperparameters across BO iterations to gain an insight into why certain kernels perform better than others. Figures 9 and 10 present the norm of the lengthscale and α for the Generalized Cauchy kernel methods across all six benchmark functions.

Lengthscale Norm We track how the lengthscale hyperparameter for the models change over time. For ARD kernels, we compute the norm of the entire lengthscale vector; for non-ARD kernel methods, we simply take the lengthscale scalar. In the Rosenbrock (300,100) case where only the first 100 of 300 input dimensions are relevant, we compute the norm using those 100 active lengthscales. Figure 9 shows a gradual increase in lengthscale norm over iterations, indicating that the GP is learning smoother functions as optimization progresses. The GCauchy variants exhibit moderate growth without extreme spikes, suggesting a stable adaptation across all the benchmarks, with Rover as an exception. We further verify that SE kernel has extreme instability due to the vast fluctuations which contribute to the poor performance in HDBO.

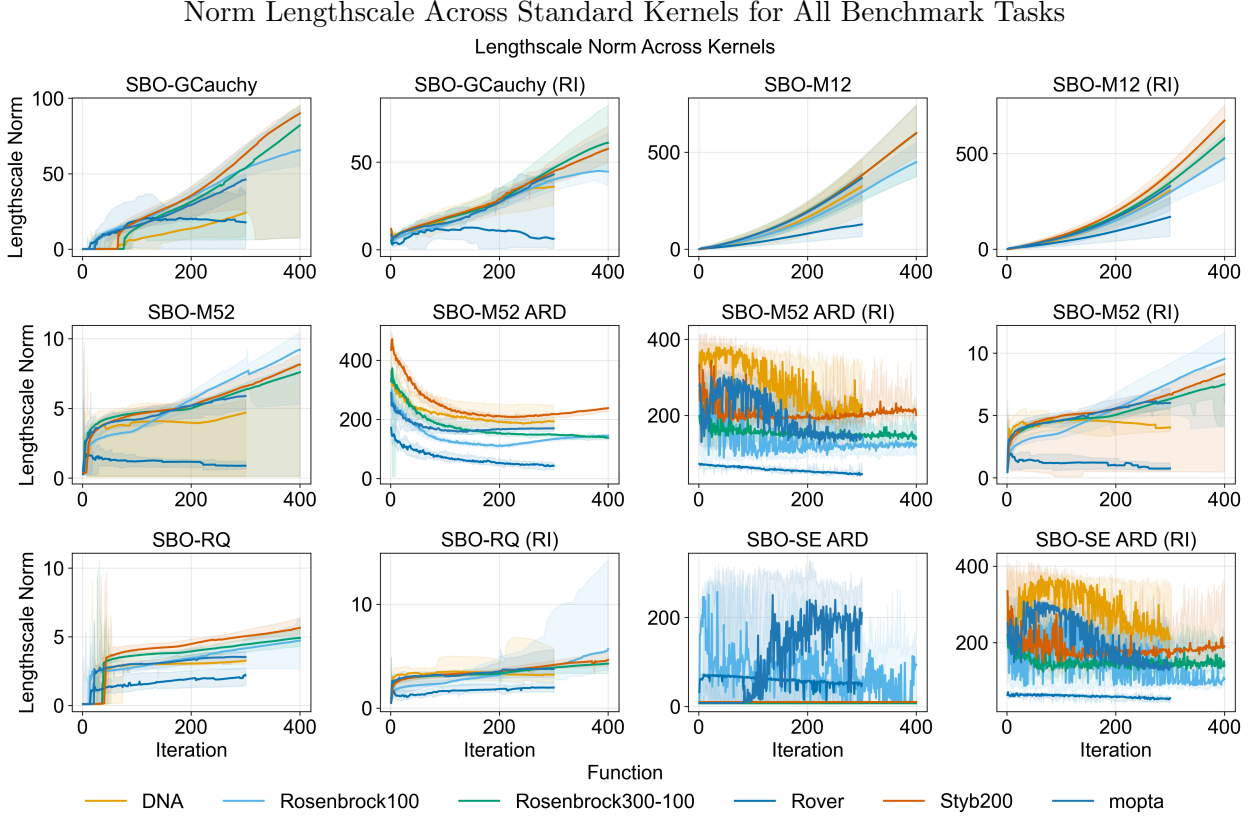


Figure 9: Norm of the lengthscale hyperparameters across iterations for different kernel types on the benchmark functions. Each subplot shows the evolution of the log lengthscale norm for a specific kernel configuration for all functions.

Generalized Cauchy Kernel α Figure 10 shows how the Generalized Cauchy kernel’s parameter α changes during BO. Recall that setting $\alpha = 2$ recovers the Rational Quadratic kernel. Across all tasks, α starts below 1 and steadily climb towards 2 (but not fully) and plateaus. This shows that the model prefers being close to RQ, but benefits from the added flexibility of the GCauchy kernel.

Generalized Cauchy Kernel β Figure 11 shows the evolution of the β parameter for the Generalized Cauchy kernel over BO iterations. Relating this to the violin plots in Figures 5 and 6, in the cases where GCauchy and GCauchy (RI) do exceptionally well we observe the β shooting

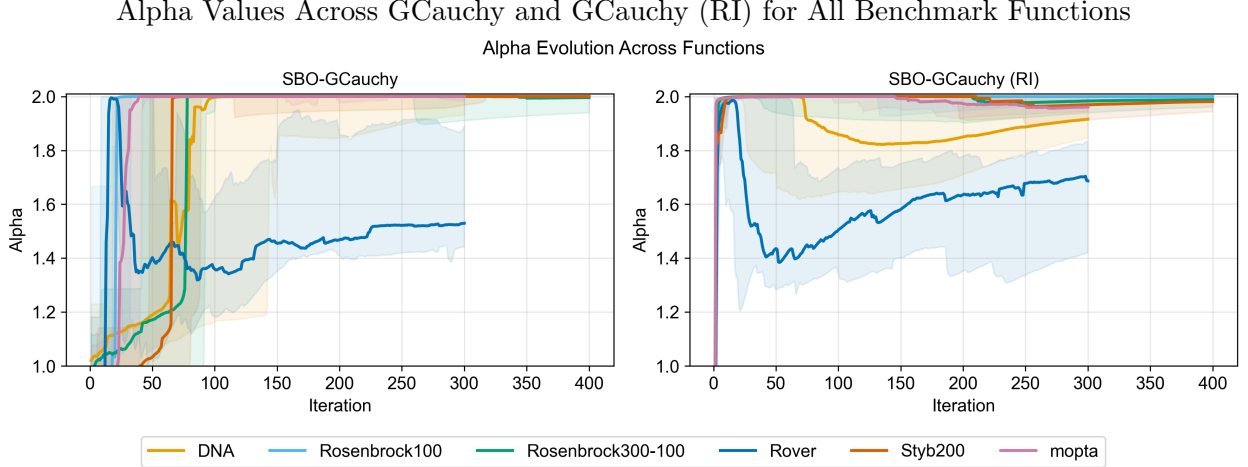


Figure 10: Alpha hyperparameter plots for GCauchy and GCauchy (RI) across iterations on the benchmark functions. Each subplot shows the evolution of the alpha value for GCauchy and GCauchy (RI) for all functions.

up. However, in the case of Rover, where Generalized Cauchy methods don't perform so well, the β value is simply low.

The GCauchy and RQ kernel methods both jump in lengthscale norm and start to converge, suggesting that the surrogate first captures the large structure and then begins fine tuning it. The Matérn kernels rise slower with more noise, suggesting a weaker identification of the underlying structure. The trends observed in Figures 9 and 11 is particularly interesting as the gradient calculations in the Appendix B for Generalized Cauchy kernel reveal that β and ℓ are both responsible to gradient vanishing; how exactly we don't know yet. The Generalized Cauchy kernel still experiences a slower rate of decay as compared to SE and Matérn (polynomial decay for GCauchy vs. exponential for SE, Matérn). Exploring the contributions of ℓ and β towards gradient decay is worth looking into and we leave it for future work.

5 Conclusion and Future Work

5.1 Conclusion

This work provides a detailed analysis of kernel choice in HDBO. We find that the Generalized Cauchy kernel works best in most settings without relying on ARD or RI. This result challenges the prior findings in Xu et al. [2], which suggest that Matérn kernels with RI are the go to for HDBO. Our findings suggest that gradient vanishing problems for standard BO in high dimensions can be mitigated by kernel choice alone. SE kernel methods are consistently outperformed even with RI and ARD, suggesting that they are not appropriate for high dimensional settings.

The results complement recent work by Xu et al. [2] and Hvarfner et al. [5], adding to the notion that standard BO can achieve competitive performance without structural modifications. The Generalized Cauchy kernel's success across most synthetic and real world tasks highlight its robustness in high dimensions.

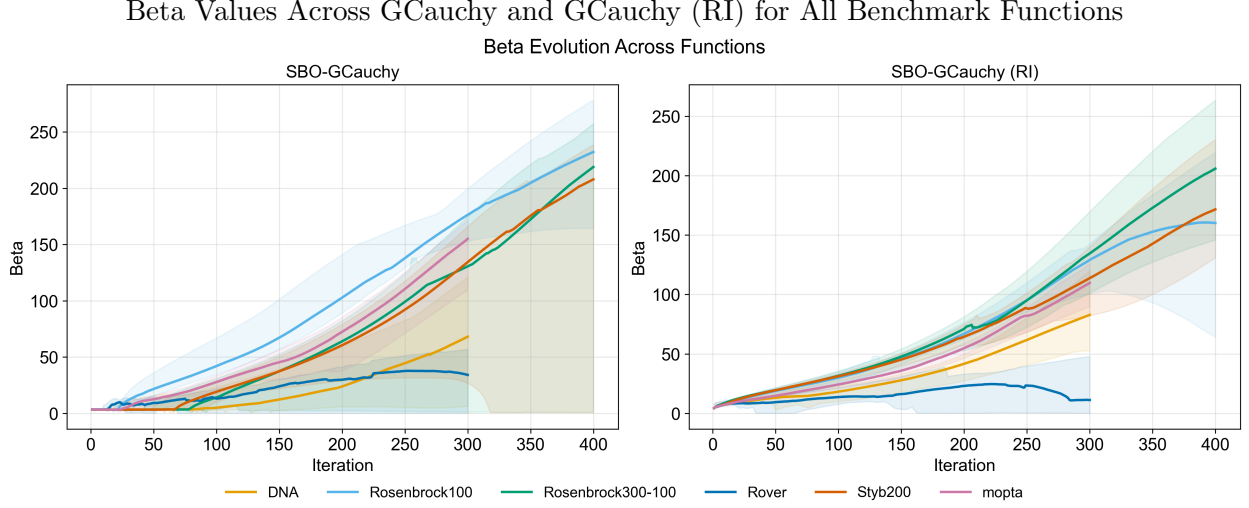


Figure 11: Beta hyperparameter plots for GCauchy and GCauchy (RI) across iterations on the benchmark functions. Each subplot shows the evolution of the beta value for GCauchy and GCauchy (RI) for all functions.

5.2 Future Work

We propose several research directions to further better our understanding of kernel choice in HDBO (maintaining the simplicity of standard BO) following this study:

- **Theoretical Analysis:** Formal investigation of Generalized Cauchy kernel’s gradient properties in high dimensional spaces.
- **Priors Specification:** Comparing how different prior specifications impact HDBO for the Generalized Cauchy kernel’s parameters
- **Likelihood Noise Setting:** Fixing the likelihood noise to small values vs. treating it as a learnable parameter.
- **Alternative Kernels:** Exploring other standard and compound kernels.

Acknowledgements

* Abhigya Koirala and Andrew Yuan are co-first authors and contributed equally to this work.

We acknowledge the generous funding and support given by the Cal Poly Pomona CAMP REU, supported by the National Science Foundation under grant number #2348984. We extend our utmost gratitude to Dr. Jimmy Risk for his invaluable guidance as our advisor throughout the program. We are also thankful to Dr. Jillian Cannons and Dr. Ryan Szypowski for leading and supervising the CAMP REU program, giving us an excellent environment for academic research and growth.

References

- [1] P. I. Frazier, “A tutorial on bayesian optimization,” *arXiv preprint arXiv:1807.02811*, 2018.
- [2] Z. Xu, H. Wang, J. M. Phillips, and S. Zhe, “Standard gaussian process is all you need for high-dimensional bayesian optimization,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=kX8h23UG6v>.
- [3] M. Köppen, “The curse of dimensionality,” in *5th online world conference on soft computing in industrial applications (WSC5)*, vol. 1, 2000, pp. 4–8.
- [4] M. Binois and N. Wycoff, “A survey on high-dimensional gaussian process modeling with application to bayesian optimization,” *ACM Transactions on Evolutionary Learning and Optimization*, vol. 2, no. 2, pp. 1–26, 2022.
- [5] C. Hvarfner, E. O. Hellsten, and L. Nardi, “Vanilla bayesian optimization performs great in high dimensions,” *arXiv preprint arXiv:2402.02229*, 2024.
- [6] D. Eriksson, M. Pearce, J. Gardner, R. D. Turner, and M. Poloczek, “Scalable global optimization via local bayesian optimization,” *Advances in neural information processing systems*, vol. 32, 2019.
- [7] A. Nayebi, A. Munteanu, and M. Poloczek, “A framework for bayesian optimization in embedded subspaces,” in *International Conference on Machine Learning*, PMLR, 2019, pp. 4752–4761.
- [8] B. Letham, R. Calandra, A. Rai, and E. Bakshy, “Re-examining linear embeddings for high-dimensional bayesian optimization,” *Advances in neural information processing systems*, vol. 33, pp. 1546–1558, 2020.
- [9] D. Eriksson and M. Jankowiak, “High-dimensional bayesian optimization with sparse axis-aligned subspaces,” in *Uncertainty in Artificial Intelligence*, PMLR, 2021, pp. 493–503.
- [10] J. K. Ziomek and H. B. Ammar, “Are random decompositions all we need in high dimensional bayesian optimisation?” In *International Conference on Machine Learning*, PMLR, 2023, pp. 43 347–43 368.
- [11] E. Han, I. Arora, and J. Scarlett, “High-dimensional bayesian optimization via tree-structured additive models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 7630–7638.
- [12] L. Papenmeier, L. Nardi, and M. Poloczek, “Bounce: Reliable high-dimensional bayesian optimization for combinatorial and mixed spaces,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 1764–1793, 2023.
- [13] L. Song, K. Xue, X. Huang, and C. Qian, “Monte carlo tree search based variable selection for high dimensional bayesian optimization,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 28 488–28 501, 2022.
- [14] Q. Nguyen, K. Wu, J. Gardner, and R. Garnett, “Local bayesian optimization via maximizing probability of descent,” *Advances in neural information processing systems*, vol. 35, pp. 13 190–13 202, 2022.
- [15] N. Hansen, “The cma evolution strategy: A comparing review,” *Towards a new evolutionary computation: Advances in the estimation of distribution algorithms*, pp. 75–102, 2006.
- [16] S. Surjanovic and D. Bingham, *Virtual library of simulation experiments: Test functions and datasets*, <http://www.sfu.ca/~ssurjano/>, Retrieved January 23, 2024, 2024.
- [17] D. R. Jones, “Large-scale multi-disciplinary mass optimization in the auto industry,” in *MOPTA 2008 Conference (20 August 2008)*, vol. 64, 2008.

- [18] Z. Wang, C. Gehring, P. Kohli, and S. Jegelka, “Batched large-scale bayesian optimization in high-dimensional spaces,” in *International Conference on Artificial Intelligence and Statistics*, PMLR, 2018, pp. 745–754.
- [19] K. Šehić, A. Gramfort, J. Salmon, and L. Nardi, “Lassobench: A high-dimensional hyperparameter optimization benchmark suite for lasso,” in *International Conference on Automated Machine Learning*, PMLR, 2022, pp. 2–1.
- [20] P. Mills, “Accelerating kernel classifiers through borders mapping,” *Journal of Real-Time Image Processing*, vol. 17, no. 2, pp. 313–327, 2020.
- [21] X. Dong and Y. Yang, “Nas-bench-201: Extending the scope of reproducible neural architecture search,” *arXiv preprint arXiv:2001.00326*, 2020.
- [22] E. Todorov, T. Erez, and Y. T. MuJoCo, “A physics engine for model-based control,” in *Proceedings of the 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033.

A Test Functions and Real-World Optimization Tasks

We evaluate the kernels on the synthetic functions defined in Xu et al. [2]. Here, d denotes the input dimension, and $d' \leq d$ is the number of effective variables (only the first d' dimensions affect the function value).

A.1 Synthetic Functions

Modified Styblinski-Tang Function

$$f(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^{d'} [(x_i - c_i)^4 - 16(x_i - c_i)^2 + 5(x_i - c_i)]$$

- Domain: $\mathbf{x} \in [-5, 5]^d$
- Parameters: c_i linearly spaced in $[0, 7.5]$ ($c_1 = 0$, $c_{d'} = 7.5$)
- Global optimum: $x_i^* = c_i - 2.903534$

Modified Rosenbrock Function

$$f(\mathbf{x}) = \sum_{i=1}^{d'-1} \left[100 \left((x_{i+1} - c_{i+1}) - (x_i - c_i)^2 \right)^2 + (1 - (x_i - c_i))^2 \right]$$

- Domain: $\mathbf{x} \in [-2.048, 2.048]^d$
- Parameters: c_i linearly spaced in $[-2, 2]$ ($c_1 = -2$, $c_{d'} = 2$)
- Global optimum: $x_i^* = c_i + 1$

Ackley Function

Defined in Surjanovic et al. [16]

$$f(\mathbf{x}) = -20 \exp \left(-0.2 \sqrt{\frac{1}{d'} \sum_{i=1}^{d'} x_i^2} \right) - \exp \left(\frac{1}{d'} \sum_{i=1}^{d'} \cos(2\pi x_i) \right) + 20 + e$$

- Domain: $\mathbf{x} \in [-32.768, 32.768]^d$
- Global optimum: $x_i^* = 0$

Hartmann-6 Function

Defined in Surjanovic et al. [16]

$$f(\mathbf{x}) = - \sum_{i=1}^4 \alpha_i \exp \left(- \sum_{j=1}^6 A_{ij} (x_j - P_{ij})^2 \right)$$

- Domain: $\mathbf{x} \in [0, 1]^d$ (only first 6 dimensions active)
- Parameters: $\alpha = [1.0, 1.2, 3.0, 3.2]$, with predefined $A_{6 \times 4}$ and $P_{6 \times 4}$ matrices
- Global optimum: $\mathbf{x}^* \approx [0.20169, 0.150011, 0.476874, 0.275332, 0.311652, 0.6573]$

A.2 Real-World Optimization Tasks

Table 4 provides descriptions for the real world benchmark tasks used in Xu et al. (2025).

Table 4: Real-World Benchmark Functions for High-Dimensional Bayesian Optimization

Benchmark Functions	Task Description	Source/Citation
Mopta08 (124)	Vehicle design problem focused on minimizing vehicle mass, subject to 68 performance constraints.	[9, 17]
SVM (388)	Hyper-parameter tuning for a kernel support vector machine regression model, optimizing 3 regularization and 385 length-scale parameters.	[9]
Rover (60)	Rover trajectory planning, aiming to find an optimal path defined by 30 waypoints in a 2D environment.	[18]
DNA (180)	Hyper-parameter optimization for weighted lasso regression on a DNA dataset, involving 180 regularization parameters.	[19, 20]
NAS201 (30)	Neural architecture search problem evaluated on the CIFAR-100 dataset.	[21]
Humanoid Standup (1003)	A novel trajectory optimization task using the MuJoCo physics engine, seeking to maximize reward for a humanoid’s stand-up actions.	[22]

B Gradient Vanishing Theory

A major problem in HDBO using standard kernels comes from *gradient vanishing*. This is observed particularly when using the SE kernel due to three major reasons [2]:

- **Distance Growth:** The pairwise distances between points grow as $\sqrt{d}$. With standard lengthscale initialization (e.g., default $\ell_0 \approx 0.693$), the normalized distance $\rho = \|\mathbf{x}_i - \mathbf{x}_j\|/\ell_0$ becomes very large.
- **Gradient Vanishing:** The gradient of the SE kernel $\partial\kappa_{\text{SE}}/\partial\ell_k \sim \rho^2 e^{-\rho^2}$ decays exponentially when $\rho > 6.58$ (for 64-bit precision). Theoretical analysis shows $P(\rho > 6.58) \rightarrow 1$ rapidly as d increases, preventing meaningful lengthscale updates.
- **Matérn Kernel Advantage:** Matérn kernels (e.g., $\nu = 5/2$) multiply exponential decay by polynomial terms, leading to slower gradient decay. So they have trainable gradients up to $\rho \approx 21.98$ under the same conditions, making them more robust in high dimensions.

B.1 Rational Quadratic Kernel

As mentioned in Section 2.2 The Rational Quadratic (RQ) kernel is defined as:

$$k_{\text{RQ}}(x, x') = \left(1 + \frac{\|x - x'\|^2}{2\alpha\ell^2}\right)^{-\alpha} = \left(1 + \frac{\rho^2}{2\alpha}\right)^{-\alpha}, \quad \rho = \frac{\|x - x'\|}{\ell}$$

Recall RQ is a special case of the Generalized Cauchy kernel (in Appendix B.2) when $\alpha = 2$.

Gradient w.r.t. ℓ Using the chain rule:

$$\frac{\partial k}{\partial \ell} = \alpha \cdot \left(1 + \frac{\rho^2}{2\alpha}\right)^{-\alpha-1} \cdot \frac{\partial}{\partial \ell} \left(\frac{\rho^2}{2\alpha}\right) = -\frac{\rho^2}{\ell} \cdot \left(1 + \frac{\rho^2}{2\alpha}\right)^{-\alpha-1}$$

As $\rho \rightarrow \infty$:

$$\frac{\partial k}{\partial \ell} \sim -\ell^{-1} \rho^{-2\alpha+2}$$

Vanishing gradient when:

$$\rho > \left(\frac{1}{\ell\xi}\right)^{1/(2\alpha)} := \tau_{\text{RQ}}^{(\ell)}$$

Gradient w.r.t. α Let $u = 1 + \frac{\rho^2}{2\alpha}$, then:

$$\frac{\partial k}{\partial \alpha} = u^{-\alpha} \cdot \left(-\log u + \frac{\rho^2}{2\alpha^2 u}\right)$$

Asymptotically:

$$\frac{\partial k}{\partial \alpha} \sim \rho^{-2\alpha} \cdot \left(-2\log \rho + \frac{1}{\alpha}\right)$$

Vanishing gradient when:

$$\rho > \left(\frac{\log \rho^2}{\xi}\right)^{1/(2\alpha)} := \tau_{\text{RQ}}^{(\alpha)}$$

B.2 Generalized Cauchy Kernel

The Generalized Cauchy kernel is defined as:

$$k(x, x') = \left(1 + \left(\frac{\|x - x'\|}{\ell}\right)^\alpha\right)^{-\beta/\alpha} = (1 + \rho^\alpha)^{-\beta/\alpha}, \quad \rho = \frac{\|x - x'\|}{\ell}, \quad \alpha \in (0, 2], \quad \beta, \ell_{\text{len}} > 0$$

If $\beta \in (0, 1]$ it has long memory with Hurst coefficient $H = \lambda/2$.

Gradient w.r.t. ℓ Using the chain rule:

$$\frac{\partial k}{\partial \ell} = \frac{\beta}{\ell} \cdot \rho^\alpha \cdot (1 + \rho^\alpha)^{-\frac{\beta}{\alpha}-1}$$

Asymptotically:

$$\frac{\partial k}{\partial \ell} \sim \ell^{-1} \rho^{-\beta}$$

Vanishing gradient when:

$$\rho > \left(\frac{\beta}{\ell \xi}\right)^{1/\beta} := \tau_{\text{GC}}^{(\ell)}$$

Gradient w.r.t. α Let $u = 1 + \rho^\alpha$, then:

$$\frac{\partial k}{\partial \alpha} = k \cdot \left[\frac{\beta}{\alpha^2} \log u - \frac{\beta}{\alpha} \cdot \frac{\rho^\alpha \log \rho}{u} \right]$$

Asymptotically:

$$\frac{\partial k}{\partial \alpha} \sim \rho^{-\beta} \cdot \log \rho$$

Vanishing gradient when:

$$\rho > \left(\frac{1}{\xi} \log \frac{1}{\xi}\right)^{1/\beta} := \tau_{\text{GC}}^{(\alpha)}$$

Gradient w.r.t. β

$$\frac{\partial k}{\partial \beta} = -\frac{1}{\alpha} \cdot \log(1 + \rho^\alpha) \cdot (1 + \rho^\alpha)^{-\beta/\alpha}$$

Asymptotically:

$$\frac{\partial k}{\partial \beta} \sim -\rho^{-\beta} \cdot \log \rho$$

Vanishing gradient when:

$$\rho > \left(\frac{1}{\xi} \log \frac{1}{\xi}\right)^{1/\beta} := \tau_{\text{GC}}^{(\beta)}$$